

AI-CRAS: AI-driven Cloud Service Requirement Analysis and Specification

Emiliano Casalicchio
Dept. of Computer Science
Sapienza University of Rome
Rome, Italy
casalicchio@di.uniroma1.it

Alberto Cotumaccio
Dept. of Computer Science
Sapienza University of Rome
Rome, Italy
cotumaccio.1852040@studenti.uniroma1.it

Abstract—Automated analysis and specification of software requirements expressed in natural language is a challenge addressed by the research community and is becoming a reality thanks to the advances in Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques. While the research community focuses mainly on generic software requirements or specialized solutions for security requirements, we find a gap in the automation of analysis and specification for requirements in the cloud computing domain and the automatic mapping of requirements on actual products offered in the cloud service market. In this research work, we propose AI-CRAS an AI-driven cloud service requirement analysis and specification methodology. The proposed method, which leverages state-of-the-art transformer-based large language model, has been implemented and validated in a real case. Experimental results demonstrate that the model performed well in binary and multi-label classification of requirements (achieving recall/F1-score of 0.96/0.92 and 0.86/0.76, respectively) and mapping requirements into actual cloud services.

Index Terms—cloud computing, cloud migration, cloud services, requirement engineering, artificial intelligence, natural language processing, cloud service broker

I. INTRODUCTION

To facilitate cloud computing adoption in the public sector (e.g. [1]) and Small Medium Enterprise (SME) sectors, it is important to provide stakeholders with tools that automate the analysis of business goals and requirements expressed in natural language, the production of technical specifications, and the mapping with the public Cloud Service Provider (CSP) offer. Automating analysis and specification of Cloud Service requirements significantly reduces cloud adoption projects' costs and time, ensuring clarity and accuracy.

Solutions for automated analysis and specification of software requirements proposed by the research community focus mainly on generic software requirements or specialized solutions for security requirements [2]. At the same time, there is a gap in the automation of analysis and specification for requirements in the cloud domain [3], [4]. Moreover, selecting appropriate cloud services remains challenging due to the market's diverse and complex offerings and heterogeneous service descriptions by major cloud vendors [5], such as Amazon Web Services, Google Cloud, and Microsoft Azure.

This research is supported by the CloudNEXTGen project funded by P.R.S. Planning Ricerche e Studi srl, in the framework of FSE REACT-EU

Requirements Engineering (RE), which stages are reported in Figure 1, represents an iterative process that deals with identifying, elaborating, structuring, specifying, analyzing, and managing requirements expressed by stakeholders in software projects [6]–[10]. Effective requirements management is critical to minimizing the risk of project failure and improving the quality of the final product. Studies in the field point out that most errors in software projects originate in the RE phase, with requirements ambiguity recognized as a major cause of failure [11]. This problematic issue originates mainly from the fact that requirements are often expressed in natural language, which increases the complexity of RE because of their intrinsic ambiguity, incompleteness, and imprecision.

The motivation for this research stems from the significant gap in the initial stage of formal documentation and analysis of user requirements in the cloud domain [3], [4]. Specifically, we investigate the following research questions:

- R1: *How can we automate the analysis and specification of cloud requirements expressed in natural language?*
- R2: *How can we automatically map customer requirements to actual cloud services offered by cloud service providers?*

We present AI-CRAS, the AI-driven methodology for Cloud

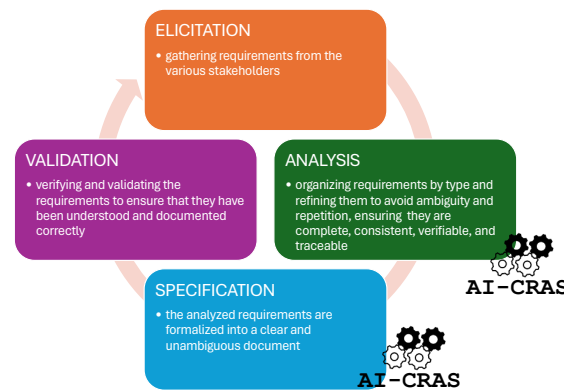


Fig. 1. Iterative Requirements Engineering Process. AI-CRAS automates the Analysis and Specification phases.

Service Requirement Analysis and Specification we designed and implemented to address the above research questions. The proposed methodology aims to forge an interdisciplinary approach by fusing insights from requirements engineering, artificial intelligence, and cloud computing to improve the accessibility, accuracy, and efficiency of cloud service selection for stakeholders.

The AI-CRAS methodology introduces the following novelties:

- a cloud service ontology covering the three main cloud service models [12] (IaaS, PaaS, and SaaS) and a wide range of cloud service types, which is missing [13] in the literature and that is fundamental for automated analysis of requirements expressed in natural language
- a publicly available dataset of cloud requirements created from open-source documents and text obtained with generative AI techniques [14], and used for AI-CRAS training and validation. *NOTE: The link provided in the references is for reviewers, it will be made public in case of paper acceptance*
- an extended BERT NLP model, trained in the Cloud Service domain, to automate the analysis and classification of requirements;
- a keyword search-based technique for automated mapping of requirement specifications into cloud services offered in the market.

The AI-CRAS framework has been validated against a set of real Cloud Service Requirements and the services offered by Amazon Web Services, Google Cloud, and Microsoft Azure. Experimental results demonstrate that the model performed well in both binary and multi-label classification of requirements (achieving recall/F1-score of 0.96/0.92 and 0.86/0.76, respectively) and in the mapping of requirements into actual cloud services.

The rest of this paper is organized as follows. Section II reviews a selection of previously conducted studies relevant to our research. Next, Section III details the methodological approach taken to address the identified problem, laying the foundation for the proposed solution. Section IV outlines the design of the solution, describing the design choices and the underlying rationale. Section V is devoted to presenting the experiments conducted to evaluate the proposed model, also including the validation phase of the methodological approach adopted. Finally, in Section VI, the conclusions of the research work are presented, highlighting obtained results and outlining possible future developments.

II. RELATED WORK

A. Languages and Tools for Requirements Engineering

The requirements engineering research community has proposed different solutions for the formal specification of requirements for software applications, e.g. [15], and for automatic code generation from requirements, e.g. [16]. However, the Model-Driven approaches proposed to simplify the management of cloud infrastructures and services do not achieve

complete automation and are primarily aimed at developers with a technical background, e.g. [3] and [4].

The ReqSpec language based on the Architecture Analysis and Design Language (AADL) [15] provides an essential formal framework for outlining the system architecture and for clear and iterative requirements specification of safety-critical real-time applications. In [16] the authors introduce a method based on Model-Driven Development to semi-automatically generate software applications using ITLingo ASL (Application Specification Language). Integrating ITLingo ASL with the low-code Quidgest Genio platform [17] makes it possible to transform detailed requirements specifications into low-code designs, enabling automatic code generation for the application.

The Argon tool [3] and the Piacere project [4] provide frameworks for high-level modeling of the final state of the desired cloud infrastructure, which can then be specialized for different cloud providers. They allow developers to focus on logic rather than low-level technical details. Using graphical notations, they allow the specification of services automatically transformed into Infrastructure as Code (IaC) scripts, e.g. Terraform. Although these projects represent significant initiatives, they assume a detailed and quantified definition of initial requirements, which is necessary to proceed with manual infrastructure modeling properly.

B. AI in Requirements Engineering

The use of AI [18] in requirement engineering aims to automate the requirements management process to solve the challenges associated with manual analysis of natural language that can be time-consuming, error-prone, and variable in interpretation. Much research has been conducted in information extraction and classification from requirements documentation using Natural Language Processing, as outlined in [2].

The first real adoption of ML in the RE was the automatic identification and classification of nonfunctional requirements (NFRs) from requirements specifications [19]. The most widely used machine learning algorithms for identifying and classifying nonfunctional requirements adopted by the research community are Naïve Bayes, Support Vector Machine and Decision Tree among supervised learning techniques, and Latent Dirichlet Allocation, K-means and Hierarchical Agglomerative Clustering as unsupervised solutions [20].

The architecture of transformers [18] has revolutionized the NLP field because of its ability to handle data sequences with an attention mechanism to capture long-range relationships. BERT [21], a transformer-based large language model pre-trained on huge text corpora, can be specialized through finetuning on specific datasets and applied it in the field of Requirement Engineering, outperforming previous methodologies [22] [23] [24].

Recent studies have investigated the applicability of ChatGPT [25], through prompt writing, in the software development cycle, highlighting its potential [26]. It was found that the model stands out for its ability to operate with high autonomy

in various activities, including requirements analysis. However, some significant limitations were identified, including a lack of traceability between phases, inconsistencies in the generated artifacts, and a sometimes excessive application of domain knowledge that can lead to unnecessary expansion of requirements.

C. Datasets

Most studies applying AI in RE rely on proprietary datasets to train their models, specifically collecting their unique sets of private documents. Scarce availability of public and open-source datasets on requirements emerges [27]. The major open-source corpora are [28], [19], [29] and [30]. The PROMISE Repository [28] includes 625 labeled requirements, both functional and non-functional. The DePaul Corpus [19], featuring 358 functional and 326 non-functional requirements, is classified into 10 categories and derived from 15 requirements specification documents. The SecReq Dataset [29] includes 511 requirements from three separate projects, divided into security-related and non-security-related requirements. The PURE Dataset [30] encapsulates about 35,000 unlabeled sentences extracted from 79 requirements documents in raw form. These requirements specifications are of different domains, have various levels of abstraction, and range from product standards to public corporate documents to academic projects. The authors offered the dataset for a wide range of uses. One particular study [22] manually labeled a subset of 7,745 sentences for fine-tuning BERT for requirements identification from unstructured documents, making the labeled dataset available and open source.

All these datasets address general software requirements and there is a distinct lack of focus on cloud computing.

D. Cloud service ontologies

According to [13] cloud computing ontologies offer solutions to specific domains and are limited and focused on certain areas, e.g. numerous ontologies have been developed to address Cloud cybersecurity compliance issues [31]. However, it is revealed that existing ontologies, in addition to being insufficiently specific, have several weaknesses, including limited coverage of cloud service types (often focused exclusively on IaaS), and a lack of definition of non-functional properties such as cloud technical characteristics.

III. PROBLEM STATEMENT AND PROPOSED METHODOLOGY

The challenge we address is the design and implementation of a methodology that allows stakeholders to focus on identifying business objectives by providing a natural language description of their desired cloud migration goals without immersing themselves in the technical complexities of cloud service selection and automates the analysis and documentation of requirements producing as result the list of cloud services needed in the migration project mapping them with the offers of cloud service providers.

A. Methodology

The proposed methodology, sketched in Figure 2, consists of the following phases:

- 1) definition of a Cloud Service Ontology
- 2) fine-tuning of the BERT model
- 3) automatic extractions of key sentences from requirement documents
- 4) automatic classification of the sentences as requirement or non-requirement
- 5) automatic mapping of the requirements on cloud services using the proposed ontology
- 6) automatic production of a specification document.

Phases 1 and 2 are performed off-line and can be repeated to introduce improvements in both the ontology and BERT model. In contrast, phases 3 – 6 are executed each time a new requirement analysis is needed.

The Cloud Service Ontology hierarchically and systematically captures the full spectrum of requirements associated with cloud computing. The ontology resulting from *phase 1* is articulated into three levels of granularity: main categories, subcategories, and specific cloud features. In *phase 2*, we fine-tuned the BERT model using a labeled dataset created by scraping public cloud online documents and exploring generative artificial intelligence methods to produce fictional documents to enrich the training set with data relevant to our scenario.

Phase 3 is addressed by proposing a text parser engine to parse user-supplied documents automatically. This tool is critical for extracting key sentences from documents in which users outline their goals related to migration to the cloud. *Phase 4*, the automatic analysis of key sentences to eliminate non-requirements and retain requirements, is based on a fine-tuned BERT model, which allows for associating each sentence with the main categories of the Cloud Service Ontology. Each sentence may refer to multiple cloud concepts. In *phase 5*, we used the keyword search technique to extract specific ontology concepts from the requirement sentences. For each identified concept, we map it to the corresponding products available in the marketplaces of major cloud service providers. Amazon Web Services [32], Google Cloud [33], and Microsoft Azure [34] have been selected for validation of our approach. Finally, in *Phase 6*, a structured requirement specification document that summarizes the results of the analysis and data extraction from the requirement document is automatically produced. Although any format can be used, we opted for a pdf document.

IV. SOLUTION DESIGN

A. Cloud Service Ontology

The goal of the Cloud Service Ontology is to categorize all requirements relevant to the cloud sphere, including both functional and non-functional requirements. The ontology is neutral with respect to cloud service providers, it is free from redundancy and consistency and has a hierarchical structure.

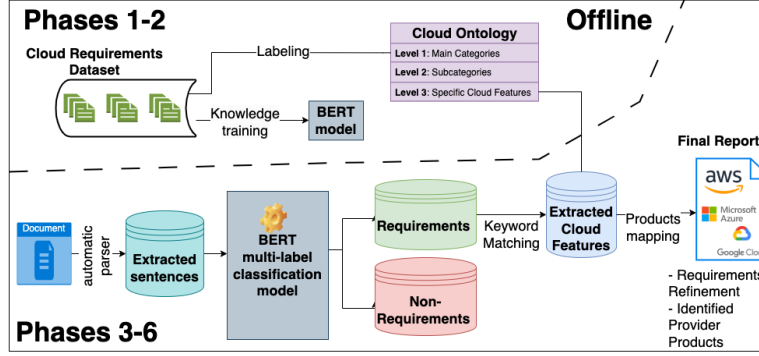


Fig. 2. Overview of the AI-CRAS methodology

TABLE I
CLOUD SERVICE ONTOLOGY OVERVIEW

Main Category	# Subcategories	# Cloud Features
Compute	4	12
Data Handling	3	5
Network	4	12
Security & Compliance	4	13
Management & Monitoring	5	16
Cloud Service Essentials	3	10

TABLE II
OVERVIEW OF DATA HANDLING CATEGORY

Subcategory	Cloud Features Requirements
Database Systems	SQL, No-SQL
Data Storage	Data Storage
Data Optimization	Cache, Disk Optimization

As outlined in Table I there are six main categories. Each category has three to five subcategories, and at the bottom level, there are the cloud features that will meet cloud requirements. For example, the category *Data Handling* is a comprehensive umbrella for all aspects of data management. Table II shows the subcategory and related Cloud feature requirements.

A limit of six main categories has been selected to maintain consistency and manageability of the multi-label classification performed with the BERT deep learning model.

B. Dataset Design

This section describes the steps to build the dataset and analyzes the dataset features.

1) *Public Documents Collection*: First, a search was conducted for public documents available on the Internet and containing requirements for cloud-related projects, e.g. migration projects. We concentrate our attention on the Request for Proposal (RFP). For each collected document, sections in which details of the cloud infrastructure or service requirements are provided were identified.

Initially, an attempt was made to extract sentences using automatic parsing and extraction engines. However, this strat-

egy was abandoned due to the poor quality of the extracted texts and the structural complexity of the parsed documents. Therefore, manual processing of the documents was opted for. It was chosen to treat each sentence as a separate data unit because the various sentence lengths and structures. Therefore, if a requirement consists of multiple sentences, it is segmented into multiple units. Sometimes, requirements are written in the form of a list with many elements. In this case, the list is transformed into a single sentence separating the various elements with commas. In some special cases, such lists have been divided into multiple sentences.

2) *Artificially Generated Documents*: To enrich our dataset, we leveraged ChatGPT (GPT-4) [35] to create texts that mimic typical requirement documents created by non-technical stakeholders planning to migrate their IT infrastructure to the cloud. This technique facilitated the automatic generation of seven concise and varied documents from which sentences were extracted. This strategy not only increases the dataset but also ensures training on data that accurately reflects the intended application context.

3) *Manual Labeling*: Each sentence in our dataset was subjected to manual examination and categorized according to the main cloud categories of the ontology, using the one-hot encoding method [36]. Manual labeling was chosen over automatic parsing to maintain high-quality data free of potential errors. To address issues such as subjective labeling bias and accuracy, the task was performed by two independent annotators, improving the reliability of the results. Labeling disagreements led to excluding the disputed text sample from the dataset, maintaining its integrity and consistency.

4) *Dataset Preprocessing*: The following preprocessing actions were taken:

- Harmonization of terminology. For example, the term "bidder", frequently recurring in RFP documents, was replaced with "provider" to ensure greater alignment with our context.
- Cleanup of imperfections. Unnecessary whitespaces and end-of periods were removed, thus ensuring no formatting disharmony and preventing possible negative impacts on

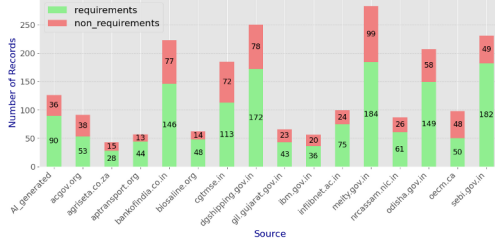


Fig. 3. Distribution of Requirements By Sources

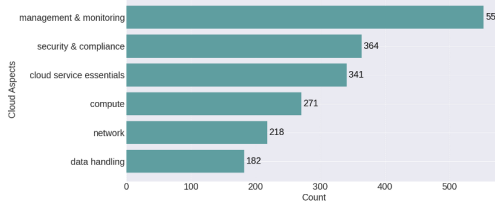


Fig. 4. Distribution of Cloud Categories

model performance.

- Removal of duplicates. The presence of duplicates can alter the training process and lead to overfitting. 103 duplicate statements were removed without distinguishing between uppercase and lowercase. For each duplicate, the first occurrence was retained.

5) *Dataset Exploration*: The cleaned dataset [14] includes 2164 entries and is structured on several columns. These include the source document from which the sentence was taken, the text of the sentence, a binary variable indicating whether the statement refers to a cloud requirement, and binary indicators for the presence or absence of each main ontology category. Requirements are the 68.1% (1474 entries), and non-requirements are 31.9% (690 entries).

Figure 3 shows the distribution of requirements (green bars) and non-requirements (red bars) broken down by source. Significant variability in the proportion among them is illustrated, also reflecting the varying magnitude of the number of extracted sentences of the documents, some with a large number and others somewhat smaller. AI-generated sentences are considered as a single document.

Figure 4 shows the frequency of the six cloud service categories. *Management & monitoring* is the most frequent requirement category, and there is no extreme imbalance among categories. We also analyzed the correlation between the requirements belonging to the ontology’s main categories. Figure 5 shows a low correlation in general. The categories corresponding to physical infrastructure requirements, *compute*, *data handling*, and *network* show some mutual correlation, suggesting a tendency of sentences to have these labels simultaneously.

Figure 6 shows the distribution of sentence lengths in the dataset. A balanced distribution, e.g., following a bell shape, is preferable as it suggests a fair variety in length without

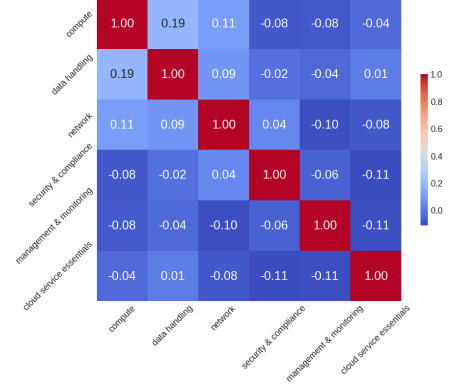


Fig. 5. Correlation Matrix of Cloud Categories

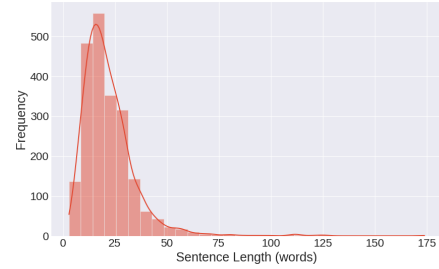


Fig. 6. Distribution of Sentence Lengths

imbalances. The analysis shows that most sentences are 0-50 words long, with a maximum concentration of around 15 to 20 words. The plot does not show the presence of any particular outliers. The average sentence length is 22 words, while the shortest sentence recorded is *24x7 technical support*, which is only 3 words long. The maximum one, consisting of 174 words, is a bulleted list turned into a single sentence.

C. Cloud Service Classification Model

Here, we illustrate the process of developing a classifier that assigns sentences to the main categories of the ontology. Considering the possibility of a single sentence belonging to several categories simultaneously, we opt for a multi-label approach [37].

1) *The multi-label classification model*: We chose the pre-trained BERT-BASE-UNCASED model to create the multi-label classification model because of its efficiency and balance between dimensionality and computational power. After analyzing our dataset, we selected the uncased model to focus on semantic understanding independently of letter case differentiation.

The pre-trained model is augmented with two additional layers: the *Dropout Layer* and the *Linear Layer*. The *Dropout Layer* introduces a regularization technique that prevents overfitting by randomly disabling some units (neurons) in the network during each training iteration. It is set a 30% probability that the output of each node in this layer will be set to zero.

The *Linear Layer* performs a linear transformation by mapping a 768-dimensional feature vector to a 6-dimensional one, corresponding to the 6 labels, using the formula $y = xA^T + b$, where: x is the input, A is the matrix of weights with size 768×6 , b is the bias vector of size 6, and y is the transformed output.

The loss function employed, `BCEWITHLOGITSLoss`, calculates the deviation between model predictions and actual labels, a strategy suitable for multi-label classification tasks, treating each label as a separate binary problem. It efficiently combines a sigmoid layer to compress outputs into a range $[0, 1]$, and binary cross entropy (BCE) in a single operation. The Adam optimizer facilitates the adjustment of weights during training, which improves the model's performance.

The model is fine-tuning on our labeled dataset.

2) *Experimental Setup for Fine-Tuning*: Tokenization, hyperparameter adjustment, and dataset partitioning are carried out as preparatory steps.

Using the WordPiece tokenizer, sentences were fragmented into smaller and manageable tokens. Special markers such as [CLS], [SEP], and [PAD] are included. This approach facilitates the model's handling of words outside its vocabulary by segmenting them into identifiable tokens, thus expanding its interpretive linguistic scope.

Key hyperparameters were subjected to iterative adjustments, culminating in a configuration of sixteen for batch size, a learning rate of $3e - 05$, a cap of 300 tokens for sequence length, and a weight decay factor of $1e - 6$. This selection aimed to balance computational requirements and model learning capacity, promoting optimal generalization without succumbing to overfitting.

The dataset was shuffled to mitigate potential bias and divided into training and validation sets, with a split of 85% and 15%, respectively, trying to obtain a representative distribution of the six target labels in both. This split was randomized using seed number 42 to ensure reproducibility.

3) *Training Implementation*: In constructing the neural network, the foundational step involves loading the pre-trained model and augmenting it with two additional layers:

- the Dropout Layer is a regularization technique that prevents overfitting by randomly disabling some units (neurons) in the network during each training iteration. It is set a 30% probability that the output of each node in this layer will be set to zero.
- the Linear Layer performs a linear transformation by mapping a 768-dimensional feature vector to a 6-dimensional one, corresponding to the 6 labels, using the formula $y = xA^T + b$, where: x is the input, A is the matrix of weights with size 768×6 , b is the bias vector of size 6, and y is the transformed output.

The loss function employed, `BCEWITHLOGITSLoss`, calculates the deviation between model predictions and actual labels, a strategy suitable for multi-label classification tasks, treating each label as a separate binary problem. It efficiently combines a sigmoid layer to compress outputs into a range $[0, 1]$, and binary cross entropy (BCE) in a single operation. The

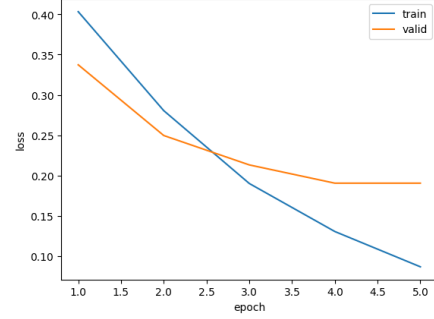


Fig. 7. Training Loss and Validation Loss

Adam optimizer facilitates the adjustment of weights during training, which improves the model's performance.

A tailored training cycle was developed, alternating training and validation phases between epochs. Each training phase involves a forward operation to derive predictions and calculate losses, followed by updates of weights by back-propagation and optimization. The subsequent validation phase evaluates the model's generalization capabilities to unseen data without weight adjustments. Validation loss serves as an indicator of the model's effectiveness outside the training dataset. Progress monitoring is achieved by tracking both training and validation losses, allowing early termination through an early stop mechanism if the validation loss stops decreasing. This allows us to avoid overfitting and ensure the generalizability of the model. As illustrated in Figure 7, the model underwent fine-tuning in five epochs.

V. EXPERIMENTS

A. Model evaluation

For each analyzed sentence, the model output can be viewed as a probability distribution among six independent labels. A threshold value of 0.20 is adopted to determine which categories to assign to a specific sentence. Categories above this threshold are considered relevant. This value was selected because it allows the model to identify relevant labels without being overly restrictive.

In the evaluation phase with the validation set, the focus is on a twofold empirical assessment of the model's performance. It begins by testing the model's efficiency in separating requirements from non-requirements. We define a requirement as any sentence the model classifies into at least one category. Next, we evaluate the model's ability to assign the correct categories to the identified requirements. We use standard evaluation metrics, including:

- **Accuracy**: is the ratio of correct predictions to the total number of predictions.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}$$

- **Precision:** is the ratio of correct positive predictions to the total number of positive predictions.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (also known as Sensitivity):** quantifies the number of correct positive predictions among all possible positive predictions.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score:** is the harmonic mean of precision and recall.

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

According to [38], automated solutions for RE tasks should ideally have a 100% recall rate. We, therefore, try to maximize this parameter to minimize the risk of missed requirements. At the same time, we seek acceptable precision to ensure we are not overwhelmed by false positives.

1) *Evaluation of Binary Classification:* The model demonstrated high performance in the binary classification of sentences between non-requirements and requirements, achieving 88% precision and 96% of recall, as shown in Table III. On the other hand, recognizing non-requirements exhibits a lower recall, a phenomenon attributable to the model's tendency to assign additional categories to sentences wrongly. However, this behavior is considered an acceptable trade-off.

TABLE III
BINARY CLASSIFICATION PERFORMANCES

Class	Precision	Recall	F1-Score
Non-Requirement	0.90	0.71	0.79
Requirement	0.88	0.96	0.92

2) *Evaluation of Multi-Label Classification:* Table IV shows the model's performance for each category. The performance recorded is high overall, underscoring the model's ability to recognize relevant instances while maintaining acceptable levels of precision in predictions. Through a macro average score of the results, which gives equal weight to each class regardless of its frequency, we show an average precision of 70%, a recall of 86%, and an F1-Score of 76%.

TABLE IV
MULTI-LABEL CLASSIFICATION PERFORMANCE

Category	Precision	Recall	F1-Score
Compute	0.60	0.86	0.70
Data Handling	0.64	0.90	0.75
Network	0.63	0.84	0.72
Security & Compliance	0.79	0.89	0.84
Management & Monitoring	0.86	0.79	0.82
Cloud Service Essentials	0.65	0.87	0.74
Macro Avg	0.70	0.86	0.76

Receiver Operating Characteristic (ROC) curve analysis was applied to each label, with the corresponding plots presented in Figure 8, including the overall average. The ROC curve represents an effective evaluation method that reflects the

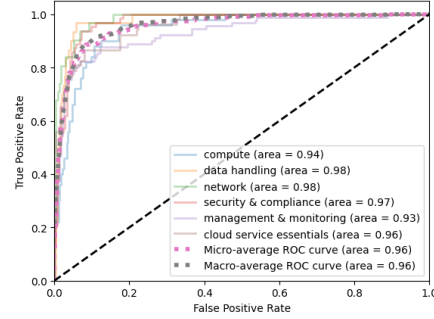


Fig. 8. ROC Curve for each category and for the micro and macro-average

performance of a classification model at various levels of decision thresholds. The Area Under the Curve (AUC) is an indicator that summarizes in a single value the ability of the model to classify over all possible threshold levels. An average AUC value of 0.96 highlights the model's ability to discriminate positive instances, minimizing the frequency of errors correctly.

B. Methodology validation

We validate the proposed methodology using a public handbook [39] to guide and support public sector authorities in purchasing cloud services, providing examples of requirements. This document was excluded from the dataset.

The stages for automated requirement analysis are: a) *parsing the user-supplied document to isolate sentences*, b) *sentence classification*, c) *cloud features extraction*, and d) *mapping to actual cloud services*.

1) *Parsing the User-Supplied Document:* In the first stage, the goal is to identify and isolate the sentences in the user-provided document, which consists of free text and paragraphs, if any. The process starts with full-text extraction and sentence segmentation. The core of sentence segmentation is Sentence Boundary Detection (SBD), which identifies the correct boundaries of each sentence. In this study, we make use of SpaCy [40], a Python library that offers a pre-trained ML model for advanced sentence segmentation.

The test document presents cloud requirements in tabular form, sometimes accompanied by comments. 173 sentences were extracted and grouped to conduct the analysis.

TABLE V
PROBABILITIES IN THE MULTI-LABEL CLASSIFICATION SETTING FOR THE SENTENCE "DATA ENCRYPTION CAPABILITIES AVAILABLE IN STORAGE AND DATABASE SERVICES"

Category	Probability
Compute	0.0413
Data handling	0.6828
Network	0.0458
Security & compliance	0.8484
Management & monitoring	0.0701
Cloud service essentials	0.0781

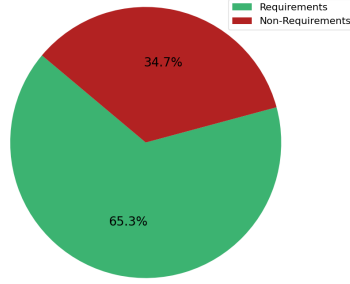


Fig. 9. Composition of Sentences in the Test Document

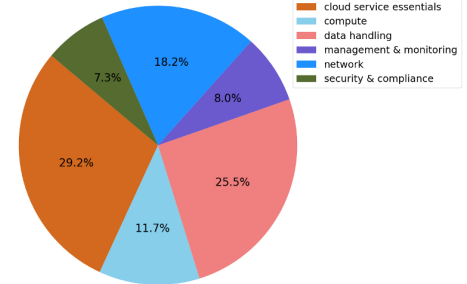


Fig. 10. Requirements Composition in the Test Document

2) *Sentences Classification*: In the second stage, the trained BERT model is used to classify the sentences extracted in the previous stage as requirement or non-requirement and assign them the ontology's macro categories.

Sentences go through a automated cleaning procedure similar to that used for creating the dataset before proceeding with the classification. Cleaning includes terminology harmonization and text correction to ensure each sentence is error-free. Next, the model is applied to each sentence through an iterative loop. The resulting output is transformed into probabilities using a sigmoid function and returned as a list of six values. Table V provides the output probabilities for the sentence:

“Data encryption capabilities available in storage and database services”

Only the categories registering a probability greater than 0.20, the predetermined threshold, are selected. The model correctly identified that the sentence deals with security and data management, as they specifically deal with encryption and the use of services such as storage and databases (will be identified in the next step).

Some sentences may not have received any category assignment; these are identified as non-requirements and, therefore, excluded from further analysis as irrelevant. Figure 9 illustrates the composition of the sentences analyzed in the test document, showing that most of them were classified by the model as cloud requirements, that is, with at least one category. Figure 10 shows how the model assigned labels to the identified requirements. Both charts are included in the final report, facilitating immediate understanding of the content and giving stakeholders a clear indication of areas of concern.

3) *Cloud Features Extraction*: This stage analyzes requirements more deeply to extract the specific cloud features defined in the ontology. For each requirement, only the cloud features contained in the associated macro categories are checked. Cloud Feature Extraction is performed using the NLP technique called Keyword Searching, based on identifying matches with a predetermined set of key terms using regular expressions. A keyword within a requirement indicates a correlation with the corresponding cloud feature.

At the base of Cloud Feature Extraction is the identification of a set of keywords, including those composed of multiple

TABLE VI
KEYWORDS MINED FOR CLOUD FEATURES SELECTED AS AN EXAMPLE

Cloud Feature	Keywords
Load Balancing	slb, load balancing, traffic distribution, load distribution, load management, load balancer, distributing traffic, lb, traffic management
NoSQL Database	key-value, graph database, non-relational database, nosql database, nosql
Firewall	firewall, packet filtering, proxy firewall, prevent access, packet inspection, traffic management, access control list, acl, inspect traffic
Encryption	encryption, cipher text, data ciphering, encrypted, encryption protocols, digital signature, key management, encryption keys, key lifecycle management, cryptographic key, kms
Dashboard	dashboard, display, management console, configuration management, self-service portal, portal, self-service portal, console, web portal, graphical user interface, gui, panel, administration interface, management interface, single interface, user interface, web Interface, online interface, webui
Multi-Region	multi-region, regional availability, cross-region, location diversity, multi-regional, availability zones, multi-site, multi-cluster, multiple locations, geographical locations

terms, and the linking of those keywords to each cloud feature. By analyzing the terminology present in the dataset, manual Term Extraction was performed to collect keywords. This process made it possible to refine the definition of new keywords progressively.

Keyword search, to perform well, requires that requirements and keywords are normalized. Three normalization steps are performed: *lowercasing*, *removal of distributing characters* and *stemming*. Lowercasing ensures that differences between upper and lowercase letters do not affect keyword searching. Disturbing characters such as “-”, and “/”, which can distort the analysis, were removed. As an example, a word such as “auto-scaling” is split into two words, “auto” and “scaling”. Punctuation has also been eliminated since signs such as commas and semicolons are no longer essential. Stemming ensures that words in different inflected forms are recognized as the same entity. It is important to note that stemming can generate “stems” that do not correspond to real words, as in the case of “database” reduced to “databas”, or “management” to “manag”. However, this heuristic approach, although aggressive, is balanced by the previous contextualization performed

TABLE VII
SAMPLE SENTENCES EXTRACTED FROM THE TEST DOCUMENT

Sentences
Provide network firewalls and web application firewall capabilities to create private networks, and control access to instances
The CISP must be certified ISO 27001
It is essential that a minimum of certifications are met
Demonstrate the ability to encrypt data in transit
The customer must be able to build or run applications in compliance with GDPR
The CISP must offer tools to alert the customer whenever a cost threshold is surpassed
Demonstrate the auto-recovery of an instance / set of instances following a failed health check
Demonstrate the automatic scaling capabilities of an application deployed behind a load balancer in a virtual compute environment
Demonstrate the ability to scale both capacity and throughput of block storage without interruption to the workload
Demonstrate a multi-region deployment of a web application including a globally replicated database
Demonstrate the ability to host containerbased workloads
Demonstrate your ability to scale from 10 to 100,000 concurrent users of the web application through dynamic auto-scaling
The CISP must be adherent to Climate Neutral Data Centre Pact
Provide the capability to implement a defence in depth strategy and thwart DDoS attacks
Demonstrate your ability to block malicious attempts to exploit the application through SQL Injection, XSS Scripting attack and other attacks
The CISP needs to be transparent as to how the application is operated and managed

by the model, which restricts the semantic range that a stem can have. Stemming is performed using the PorterStemmer algorithm from the NLTK library [41].

Table VI shows the keywords associated with some cloud features. This assignment was made for each cloud feature defined in the ontology, of which there are 68. It should be noted that each feature has only one main category and subcategory to which it belongs, which are not mentioned due to space limitations.

4) *Requirements Refinement*: After extracting cloud features, it is essential to illustrate the results to the user through precise requirements refinement, which consists of organizing and specifying the information gathered to make it clear and easily understood. This refinement process, which is automated to ensure efficiency and accuracy, is illustrated through Table VII, which shows a sample of the sentences in a test document, and in Figure 11, which shows a portion of the requirements refinement output report, generated automatically, for the test document.

Following Requirement Engineering standards, we adopted a structured "forest"-like representation, organized in a modular manner. The specification is divided into a numbered list: each top-level element corresponds with the six main categories of the ontology, representing core concepts. For the sake of space, only the first three macro-categories are shown. Within these categories, sublists detail the various subcategories established in the second layer of the ontology. For each of these, cloud features identified through keyword matching are listed, including frequencies of occurrence for each feature, enclosed in brackets.

5) *Mapping to Cloud Products*: In the final stage, the key objective is to associate each cloud feature with the specific products offered by the cloud service providers we consider. A detailed analysis was conducted on the CSP markets for each cloud feature to identify the products that best represent it. Some cloud features have a one-to-one match and others match a possible group of products.

Figure 12 lists the products of three CSPs that match the cloud features previously identified. Only mappings within the main categories *compute*, *data handling*, *network* and *security and compliance* are shown because page limit. It is important to note that some features represent non-functional requirements, such as "Latency" for the Network category, and therefore, a mapping could not be established.

- 1) The following requirements were extracted for main category "Compute":
 - 1.1) Cloud features for subcategory "Resource types" are listed below:
 - 1.1.1) Virtual Machine (7)
 - 1.1.2) Containerization (3)
 - 1.1.3) Serverless (2)
 - 1.2) Cloud features for subcategory "Elasticity" are listed below:
 - 1.2.1) Scaling (8)
 - 1.2.2) Load Balancing (4)
 - 1.3) Cloud features for subcategory "Host provisioning" are listed below:
 - 1.3.1) Auto-Provisioning (1)
- 2) The following requirements were extracted for main category "Data handling":
 - 2.1) Cloud features for subcategory "Data storage" are listed below:
 - 2.1.1) Data Storage (5)
 - 2.2) Cloud features for subcategory "Database systems" are listed below:
 - 2.2.1) Relational Database (4)
 - 2.2.2) NoSQL Database (2)
- 3) The following requirements were extracted for main category "Network":
 - 3.1) Cloud features for subcategory "Infrastructure" are listed below:
 - 3.1.1) Local Network (6)
 - 3.2) Cloud features for subcategory "Services" are listed below:
 - 3.2.1) Firewall (3)
 - 3.3) Cloud features for subcategory "Metrics" are listed below:
 - 3.3.1) Latency (1)

Fig. 11. Portion of Refinement of Cloud Requirements output report obtained from the test document

VI. CONCLUSIONS

The proposed methodology simplifies cloud service requirements' analysis, specification, and selection of actual cloud services from selected CSPs by leveraging cutting-edge Deep Learning and Natural Language Processing techniques. By transforming stakeholder needs, articulated in natural language, into precise and structured technical specifications, AI-CRAS bridges the gap in automated analysis and specification of requirements for the cloud domain. Our results affirm the AI-CRAS method's effectiveness, particularly its superior capability in identifying and classifying sentences (fine-tuning the BERT model on the cloud domain) and achieving robust performance, e.g., the recall for binary classification (requirements/non-requirements) is 0.96, and the AUC values for macro and micro-average is 0.96.

Category: Compute Feature: Virtual Machine AWS: EC2 Google: Compute Engine Azure: Virtual Machines Feature: Containerization AWS: ECS, EKS Google: Kubernetes Engine Azure: Kubernetes Service (AKS) Feature: Serverless AWS: Lambda Google: Cloud Functions Azure: Functions Feature: Scaling AWS: EC2 Auto Scaling, EC2 Google: Cloud AutoScaling, Compute Engine Azure: Virtual Machine Scale Sets, Virtual Machines Feature: Load Balancing AWS: ELB, EC2 Google: Cloud Load Balancing, Compute Engine Azure: Load Balancer, Virtual Machines Feature: Auto-Provisioning AWS: CloudFormation Google: Deployment Manager Azure: Resource Manager	Category: Data handling Feature: Data Storage AWS: S3 (Simple Storage Service) Google: Cloud Storage Azure: Blob Storage Feature: Relational Database AWS: RDS, Aurora Google: Cloud SQL Azure: SQL Database Feature: NoSQL Database AWS: DynamoDB Google: Firestore Azure: Cosmos DB	Category: Network Feature: Local Network AWS: VPC Google: VPC Azure: Virtual Network Feature: Firewall AWS: Network Firewall Google: Cloud Firewall Azure: Firewall	Category: Security & compliance Feature: Core IAM AWS: IAM Google: Cloud IAM Azure: Azure Active Directory (AD) Feature: Data Security AWS: Macie Google: Sensitive Data Protection Azure: Azure Information Protection Feature: Encryption AWS: Key Management Service Google: Cloud Key Management Service Azure: Azure Key Vault Feature: Confidentiality AWS: Secrets Manager Google: Secret Manager Azure: Azure Key Vault (Secrets) Feature: Vulnerability Assessment AWS: Inspector Google: Security Command Center Azure: Azure Security Center Feature: Threat Detection AWS: GuardDuty Google: Security Command Center Azure: Azure Security Center Feature: Cyber-Attack Protection AWS: Shield Google: Cloud Armor Azure: Azure DDoS Protection
--	---	--	---

Fig. 12. Mappings to vendor products

The keyword search technique further enables extracting cloud-specific features from the identified requirements. In addition, a new approach for dataset compilation was introduced, using generative artificial intelligence technologies to greatly enrich the quality and diversity of training data.

Several avenues for further research have been identified. Improving ontology and keyword collection could refine cloud feature extraction while customizing the document parsing phase with a specially trained model could make sentence extraction more accurate. Future work could also explore handling requirements that do not match existing keywords for cloud features. One potential strategy involves using vector embeddings from models such as BERT to determine semantic similarities between sentences and keywords, thus facilitating more accurate matches. In addition, testing the methodology with requirement documents from real projects and soliciting feedback from potential end users, such as organizations considering cloud migration, could provide valuable insights for refinements.

REFERENCES

- [1] "Strategia cloud italia: Gli indirizzi strategici per la pubblica amministrazione," <https://innovazione.gov.it/dipartimento/focus/strategia-cloud-italia/>, Dipartimento per la trasformazione digitale Italia, 2021.
- [2] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E. Chioasca, and R. T. Batista-Navarro, "Natural language processing (NLP) for requirements engineering: A systematic mapping study," *CoRR*, vol. abs/2004.01099, 2020. [Online]. Available: <https://arxiv.org/abs/2004.01099>
- [3] J. Sandobalín, E. Insfran, and S. Abrahão, "Argon: A model-driven infrastructure provisioning tool," in *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 2019, pp. 738–742.
- [4] E. Osaba, J. Diaz-de Arcaya, L. Orue-Echevarria, J. Alonso, J. L. Lobo, G. Benguria, and I. n. Etxaniz, "Piacere project: description and prototype for optimizing infrastructure as code deployment configurations," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, ser. GECCO '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 71–72. [Online]. Available: <https://doi.org/10.1145/3520304.3533938>
- [5] F. Nawaz, A. Mohsin, and N. Janjua, "Service description languages in cloud computing: state-of-the-art and research issues," *Service Oriented Computing and Applications (SOCA)*, vol. 13, pp. 109–125, 2019. [Online]. Available: <https://doi.org/10.1007/s11761-019-00263-z>
- [6] I. Sommerville, *Software Engineering*, 9th ed. Saddle River: Pearson Education Inc., 2011.
- [7] "Ieee recommended practice for software requirements specifications," *IEEE Std 830-1998*, pp. 1–40, 1998.
- [8] M. Tasnim, M. Rayhan, Z. Zhang, and T. Poranen, "A systematic literature review on requirements engineering practices and challenges in open-source projects," in *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, 2023, pp. 278–285.
- [9] B. Tenbergen and M. Daun, "Is requirements-engineering research delivering what it promised?: A review of its accomplishments and opportunities after 10 years," *IEEE Software*, vol. 36, no. 4, pp. 6–11, 2019.
- [10] B. H. Cheng and J. M. Atlee, "Research directions in requirements engineering," in *Future of Software Engineering (FOSE '07)*, 2007, pp. 285–303.
- [11] A. Hussain and E. O. C. Mkpojiogu, "Requirements: Towards an understanding on why software projects fail," *AIP Conference Proceedings*, vol. 1761, no. 1, p. 020046, 08 2016. [Online]. Available: <https://doi.org/10.1063/1.4960886>
- [12] F. Liu, J. Tong, J. Mao, R. Bohn, J. Messina, M. Badger, and L. D., "Nist cloud computing reference architecture," 2011-09-08 2011.
- [13] J. Agbaegbu, O. T. Arogundade, S. Misra, and R. Damaševičius, "Ontologies in cloud computing—review and future directions," *Future Internet*, vol. 13, no. 12, 2021. [Online]. Available: <https://www.mdpi.com/1999-5903/13/12/302>
- [14] A. Cotumaccio and E. Casalicchio, "Ai-cras dataset," <https://gitfront.io/r/cotumaccio/Sx5CKo4ALwJr/AI-CRAS-Dataset/>, 2024.
- [15] P. Feiler, D. Gluch, and J. Hudak, "The architecture analysis and design language (aadl): An introduction," February 2006.
- [16] P. Galhardo and A. R. d. Silva, "Combining rigorous requirements specifications with low-code platforms to rapid development software business applications," *Applied Sciences*, vol. 12, no. 19, 2022. [Online]. Available: <https://www.mdpi.com/2076-3417/12/19/9556>
- [17] Quidgest, "Genio," <https://genio.quidgest.com>, online; Accessed April 2024.
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017.
- [19] J. Cleland-Huang, R. Settimi, X. Zou *et al.*, "Automated classification of non-functional requirements," *Requirements Engineering*, vol. 12, pp. 103–120, 2007. [Online]. Available: <https://doi.org/10.1007/s00766-007-0045-1>
- [20] M. Binkhonain and L. Zhao, "A review of machine learning algorithms for identification and classification of non-functional requirements," *Expert Systems with Applications: X*, vol. 1, p. 100001, 2019.
- [21] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.
- [22] V. Ivanov, A. Sadovykh, A. Naumchev, A. Bagnato, and K. Yakovlev, "Extracting software requirements from unstructured documents," 2022.
- [23] S. Bashir, M. Abbas, M. Saadatmand, E. P. Enoui, M. Bohlin, and P. Lindberg, "Requirement or not, that is the question: A case from the railway industry," in *Requirements Engineering: Foundation for Software Quality*, A. Ferrari and B. Penzenstädler, Eds. Cham: Springer Nature Switzerland, 2023, pp. 105–121.
- [24] A. Sainani, P. R. Anish, V. Joshi, and S. Ghaisas, "Extracting and classifying requirements from software engineering contracts," in *2020 IEEE 28th International Requirements Engineering Conference (RE)*, 2020, pp. 147–157.
- [25] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020.
- [26] K. Dae-Kyoo, "Using chatgpt to develop software systems: Alert to software engineers?" April 2023, lab: Dae-Kyoo Kim's Lab. [Online]. Available: <https://doi.org/10.13140/RG.2.2.26388.78725>
- [27] C. Cheliger, J. Huang, G. Wu, N. Bhuiyan, Y. Xu, and Y. Zeng, "Machine learning in requirements elicitation: a literature review," *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, vol. 36, p. e32, 2022.

- [28] J. Sayyad Shirabad and T. Menzies, "The PROMISE Repository of Software Engineering Databases." School of Information Technology and Engineering, University of Ottawa, Canada, 2005. [Online]. Available: <http://promise.site.uottawa.ca/SERepository>
- [29] E. Knauss, S. Houmb, K. Schneider, S. Islam, and J. Jürjens, "Supporting requirements engineers in recognising security issues," in *Requirements Engineering: Foundation for Software Quality*, D. Berry and X. Franch, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 4–18.
- [30] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "Pure: A dataset of public requirements documents," in *2017 IEEE 25th International Requirements Engineering Conference (RE)*, 2017, pp. 502–505.
- [31] V. Singh and S. K. Pandey, "A comparative study of cloud security ontologies," in *Proceedings of 3rd International Conference on Reliability, Infocom Technologies and Optimization*, 2014, pp. 1–6.
- [32] Amazon, "Aws marketplace," <https://aws.amazon.com/marketplace>, online; Accessed April 2024.
- [33] Google, "Google cloud marketplace," <https://cloud.google.com/marketplace>, online; Accessed April 2024.
- [34] Microsoft, "Azure marketplace," <https://azuremarketplace.microsoft.com/>, online; Accessed April 2024.
- [35] OpenAI *et al.*, "Gpt-4 technical report," OpenAI, Technical Report, 2023.
- [36] J. Hancock and T. Khoshgoftaar, "Survey on categorical data for neural networks," *Journal of Big Data*, vol. 7, no. 28, 2020. [Online]. Available: <https://doi.org/10.1186/s40537-020-00305-w>
- [37] N. Endut, W. Hamzah, I. Ismail, M. Yusof, Y. Abu Baker, and H. Yusoff, "A systematic literature review on multi-label classification based on machine learning algorithms," *TEM Journal*, vol. 11, pp. 658–666, 2022. [Online]. Available: <https://doi.org/10.18421/tem112-20>
- [38] D. Berry, "Empirical evaluation of tools for hairy requirements engineering tasks," *Empirical Software Engineering*, vol. 26, p. 111, 2021. [Online]. Available: <https://doi.org/10.1007/s10664-021-09986-0>
- [39] *Buying Cloud Services in Public Sector*, Cloud Infrastructure Services Providers in Europe (CISPE), Feb. 2022, handbook - Including Sample RFP Language for a Cloud Framework Agreement, Version 2.
- [40] "Spacy," <https://spacy.io/>, online; Accessed April 2024.
- [41] "Nltk," <https://www.nltk.org/>, online; Accessed April 2024.